

PROJECT MANAGEMENT IMPROVING SOFTWARE ECONOMICS

1

Vivek Gupta
Asstt. Prof. Dept of CSE

4. Software Economics Improvement Trends

Cost Model Parameters	Trends
Size Abstraction and component based development technologies	Higher-order languages (C++, Ada 95), Object-oriented (analysis, design, programming), reuse, commercial components
Process Methods and techniques	Iterative development, process maturity levels, architecture first development, acquisition reform
Personnel People factors	Training and personnel skill development, teamwork, win-win conditions
Environment Automation technologies and tools	Integrated tools (visual modeling, compiler, editor, debugger, change management), open systems, hardware platform performance, automation of coding, documentation, testing, analysis
Quality Performance, quality, accuracy	Hardware platform performance, demonstration-based assessment, statistical quality control

21/02/2020

FIVE ECONOMY IMPROVEMENT PARAMETERS

- Reducing the size or complexity of what needs to be developed.
- Improving the development process.
- Using more-skilled personnel and better teams (not necessarily the same thing).
- Using better environments (tools to automate the process).
- Trading off or backing off on quality thresholds.

01/02/2020

REDUCING SOFTWARE SIZE

- Languages.
- Object Oriented Methods and Visual Modeling.
- Reuse
- Commercial Components.

LANGUAGE COMPARISON

- Universal function points and SLOC metrics are useful estimators for software after a candidate solution is formulated and an implementation language is known.
- Languages expressiveness of some of today's popular languages :

Language	SLOC per UFP
Assembly	320
C	128
Fortran 77	105
COBOL 85	91
Ada 83	71
C++	56
Ada 95	55
Java	55
Visual Basic	35

OBJECT ORIENTED METHOD & VISUAL MODELING

- Better capture of software abstractions leads to better communications and better teamwork.
- Continuous integration leads to earlier risk recognition and less costly corrections.
- Object-oriented architectures provide better separation of disparate elements of a system and help create firewalls for less costly development.
- Object-oriented and visual modeling create a strong architectural vision for cleaner, less-costly products.

01/02/2020

OBJECT ORIENTED APPROACH CONTINUES..

- Object-oriented programming languages appear to benefit both software productivity and software quality.
- The fundamental impact of object-oriented technology is in reducing the overall size of what needs to be developed.
- An object-oriented model of the problem and its solution encourages a common vocabulary between the end users of a system and its developers.
- An object-oriented architecture provides a clear separation of concerns among disparate elements of a system.

01/02/2020

REUSE OF SOFTWARE

- Common architectures.
- Development environments.
- Operating systems.
- Database Management systems.
- Networking products.
- Office applications.

REUSE OF SOFTWARE

- Reusing existing components and building reusable components have been natural software engineering activities since the earliest improvements in programming languages.
- With reuse in order to minimize development costs while achieving all the other required attributes of performance, feature set, and quality.
- Most truly reusable components of value are transitioned to commercial products supported by organizations with the following characteristics:
 - They have an economic motivation for continued support.
 - They take ownership of improving product quality, adding new features, and transitioning to new technologies.
 - They have a sufficiently broad customer base to be profitable.

COMMERCIAL COMPONENTS

- The use of commercial components is certainly desirable as a means of reducing custom development.
- Following Table identifies some of the advantages and disadvantages of using commercial components.

01/02/2020

Approach	Advantages	Disadvantages
Commercial Components	• Predictable License Costs • Broadly used Mature Technology • Hardware Software Independence • Rich in Functionality	• Frequent Upgrades • Upfront License Fees • Recurring maintenance fees. • Dependency on vendors. • Runtime Efficiency sacrifices. • Functionality Constraints. • Integration not always trivial • No control over upgrades and maintenance. • Unnecessary features that consume extra resources. • Often inadequate reliability and stability.

COMMERCIAL COMPONENTS CONTINUES..

Approach	Advantages	Disadvantages
Custom Development	•Complete Change Freedom. •Smaller, often simpler implementation. •Often better performance. •Control of Development and enhancement.	•Expensive, unpredictable Development. •Unpredictable availability date. •Undefined maintenance model. •Often immature and fragile. •Single platform dependency. •Drain on expert resources.

01/02/2020

IMPROVING SOFTWARE PROCESS

- Development projects are complex undertakings
 - Some activities done in parallel; others sequential.
 - Some activities: overhead; some production.
 - Production activities → the project itself – requirements elicitation and modeling, analysis, design, implementation...
 - Overhead activities → planning, progress monitoring, risk assessment, financial assessments, configuration control, quality assessment, integration, testing, late rework, management, personnel training, etc....
 - ➔ Objective: minimize overhead and direct these energies toward production activities.

IMPROVING SOFTWARE PROCESS CONT...

- It is all about process!
- A high-quality process reduces:
 - Required effort and thus schedule
 - Project time yet with improved quality.
- Three generic improvement scenarios:
 - Improve efficiency of each step in process
 - Eliminate some steps in the process
 - Undertake the same number of steps, but employ concurrency where possible

IMPROVING SOFTWARE PROCESS CONT...

- First approach
 - more efficient steps – older approach - is good,
 - more ROI on second (fewer steps) and
 - third (concurrency in activities)
- We want the highest quality system with fewest iterations in least time.
 - Must eliminate rework and late scrap – fixing things up!
 - Integration testing is culprit.
 - Fixing things up as a result of integration and system tests are killers!!!
- Our Process must reduce the probability of late rework!

IMPROVING TEAM EFFECTIVENESS

- Teamwork is much more important than the sum of the individuals.
- Some maxims of team management include the following:
- A well-managed project can succeed with a nominal engineering team.
- A mismanaged project will almost never succeed, even with an expert team of engineers.
- A well-architected system can be built by a nominal team of software builders.
- A poorly architected system will flounder even with an expert team of builders.

01/02/2020

TEAM EFFECTIVENESS CONT...

- Boehm five staffing principles are:
 - The principle of top talent: Use better and fewer people.
 - The principle of job matching: Fit the tasks to the skills and motivation of the people available.
 - The principle of career progression: An organization does best in the long run by helping its people to self-actualize.
 - The principle of team balance: Select people who will complement and harmonize with one another.
 - The principle of phase-out: Keeping a misfit on the team doesn't benefit anyone

01/02/2020

IMPROVING AUTOMATION THROUGH SOFTWARE ENVIRONMENT

- Many software tools available on the market which automate the development and maintenance of artifacts.
- Common Tools:
 - Planning Tools (Microsoft Project)
 - Requirement Management Tools (Visio, Rose)
 - Visual Modeling Tools (Visio, Rational Rose, ErWin Case Tool, Oracle Designer)
 - Compilers, Editors, Debuggers (Visual Studio, Oracle JDeveloper, Jbuilder)
 - Quality Assurance Tools
- Impact of tools and automation generally allows improvements
- Tools and environments must be viewed as the primary delivery vehicle for process automation and improvement, so their impact can be much higher.

MODERN APPROACH

- 1st artifact – Define the development and maintenance environment and select a set of tool which will automate the process
- Development Environment Should Support
 - Requirement Management
 - Document Automation
 - Host/Target Programming Tools
 - Testing
 - Feature/Defect Tracking •
- Tool should allow designer/developer to easily traverse between the various artifacts

IMPORTANT CONCEPTS

- Forward Engineering – automation of one artifact from another more abstract representation.
(Class Diagrams -> Source Code)
- Reverse Engineering – generation of a more abstract representation from an existing artifact
(Source Code -> Class Diagram)
- Round Trip Engineering – Forward & Reverse Engineering
 - Describes an environment in which once one artifact is modified, all the other artifacts both preceding and following it are modified as well.
 - Great in theory, but very hard to simulate in real world

WHY USE TOOLS ?

- “Isolated impact of tools and automation allows improvement in 20% – 40% in effort” (Reiss)
- Vendor claims are too simplistic and should be disregarded
- “It’s unlikely that any individual tool will improve a project’s productivity by more than 5%”
- Usually the benefits of using a tool are not realized unless there is some sort of change in process.
- Process is most important and software environments should be viewed and used as primary delivery vehicles for process automation and improvement.
- Testing tool will improve your testing productivity by 20%. Because test activities consume 50% of the life cycle, there will be a 10% net productivity gain to the entire project.

SUMMARY

- Requirements analysis and evolution activities consume 40% of life-cycle costs.
- Software design activities have an impact on more than 50% of the resources.
- Coding and unit testing activities consume about 50% of software development effort and schedule.
- Test activities can consume as much as 50% of a project's resources.
- Configuration control and change management are critical activities that can consume as much as 25% of resources on a large-scale project.
- Documentation activities can consume more than 30% of project engineering resources.
- Project management, business administration, and progress assessment can consume as much as 30% of project budgets.

PRINCIPLES OF MODERN SOFTWARE MANAGEMENT

- Top 10 principles of modern software management are:

1. Base the process on an architecture-first approach:

- This requires that a demonstrable balance be achieved among the driving requirements, the architecturally significant design decisions, and the lifecycle plans.

2. Establish an iterative life-cycle process that confronts risk early:

- an iterative process that refines the problem understanding, an effective solution, and an effective plan over several iterations encourages a balanced treatment of all stakeholder objectives.

3. Transition design methods to emphasize component-based development:

- Moving from a line-of-code mentality to a component-based mentality is necessary to reduce the amount of human-generated source code and custom development.

PRINCIPLES CONTD..

4. Establish a change management environment.

- The dynamics of iterative development, including concurrent workflows by different teams working on shared artifacts, necessitates objectively controlled baselines.

5. Enhance change freedom through tools that support round-trip engineering.

- Round-trip engineering is the environment support necessary to automate and synchronize engineering information in different formats(such as requirements specifications, design models, source code, executable code, test cases).

6. Capture design artifacts in rigorous, model-based notation.

- A model based approach (such as UML) supports the evolution of semantically rich graphical and textual design notations.

PRINCIPLES CONTD...

7. Instrument the process for objective quality control and progress assessment.

- Life-cycle assessment of the progress and the quality of all intermediate products must be integrated into the process.

8. Use a demonstration-based approach to assess intermediate artifacts.

9. Plan intermediate releases in groups of usage scenarios with evolving levels of detail.

- It is essential that the software management process drive toward early and continuous demonstrations within the operational context of the system, namely its use cases.

10. Establish a configurable process that is economically scalable.

- No single process is suitable for all software developments.